KOTLIN ANDROID · FULL-STACK PRODUCT BRIEF

ARTMUSEUM

Real People · Real Thoughts · A Kinder World

A modern, art-inspired social app for sharing the thoughts, experiences, and artwork that make life meaningful.

PRODUCT NORTH STAR

Every post is a small exhibit from a real life. Artmuseum helps people share honestly, discover perspectives, and make room for peace, courage, love, realism, and growth—without turning human experiences into a competition.

> *Build a real, working Android application—not a static mockup, demo feed, or simulated backend.*

1. Product Overview

Artmuseum is a native Android social platform built with Kotlin. Users authenticate, create profiles, publish long-form thoughts with optional artwork and captions, browse a real-time feed, like posts, save posts, and receive activity notifications. The experience should feel like a quiet contemporary art museum rather than a noisy conventional social network.

Product principles

- Human-first: prioritize sincere stories, thoughtful reading, and respectful discussion.
- Artful, not ornamental: visual beauty supports content and never overwhelms it.
- Real functionality: every visible control must perform its stated action against Firebase or local system APIs.
- Mobile-native: compact layouts, thumb-friendly controls, responsive typography, and smooth scrolling.
- Trust and safety: report, block, delete own posts, account deletion, and privacy controls.

2. Technology Stack

Android client

- Kotlin, Jetpack Compose, Material 3 with a custom design system.
- MVVM with repository pattern; Kotlin Coroutines and Flow for asynchronous and real-time state.
- Navigation Compose; Hilt for dependency injection where compatible with the chosen toolchain.
- Firebase BoM with Firebase Authentication, Cloud Firestore, Cloud Storage, and Firebase Cloud Messaging.
- Coil for image loading; Android Photo Picker for selecting images; Activity Result APIs for system permissions.
- Firebase Emulator Suite for development and tests—not as a production substitute.

Project configuration

- Suggested application ID: com.artmuseum.app.
- Use current stable compatible Kotlin, Android Gradle Plugin, Compose BOM, and Firebase SDK versions; pin and document them.
- Suggested minSdk: API 26 unless older-device support is required.
- Keep secrets out of source control. Firebase client configuration is not a substitute for security rules.

3. Authentication & First Launch

- Welcome screen with Artmuseum wordmark, circular art-inspired emblem, short tagline, and Continue with Google / Continue with Email.
- Google sign-in using Firebase Authentication and the recommended current Android credential flow.
- Email registration, email/password sign-in, password reset, email verification state, sign-out, and session restoration.
- New accounts go to profile setup; completed profiles go to Home.
- Show loading, retry, validation, and authentication error states. Never fake successful sign-in.
- Optional guest browsing must be clearly distinguished; write actions should request sign-in.

4. Navigation & Screen Map

- Home: real-time feed with For You, Life, Relationships, Courage, Growth, Peace, and Art filters.
- Explore: discover posts and themes; search posts, topics, and users with a documented search implementation.
- Create: long-form post composer with optional image, caption, and topic tags.
- Saved: posts bookmarked by the current user, synced to Firestore.

- Profile: personal details, posts, edit profile, and settings.
- Notifications: activity list with unread state and links to the relevant post or profile.

Use compact bottom navigation: Home, Explore, Create, Saved, Profile. Notifications are accessible from a small top-bar icon with an unread indicator.

5. Core Social Features

Create posts

- Support text-only posts and posts with an artwork image in the initial release; design the model to allow galleries later.
- No artificial word count in the editor. Handle long text with scalable UI and backend-safe validation; show a helpful error if a platform limit is reached.
- Optional image caption, topic tags, preview, upload progress, retry, and cancel.
- Upload image to Firebase Storage, then create the Firestore post after upload succeeds. Clean up orphaned uploads where feasible.
- Allow authors to edit/delete their own posts and confirm destructive actions.

Feed and post detail

- Show author avatar/name, timestamp, text, artwork, caption, topic chips, like count, and save control.
- Paginate feed queries; do not download the entire collection.
- Use Firestore snapshot listeners for live updates and remove listeners when screens leave scope.
- Make like/unlike idempotent; store a per-user like record to prevent duplicate likes.
- Save/unsave must persist across devices and sessions.
- Post detail supports full text, image viewing, and a clear return path.

Comments

Recommended for the initial product: a paginated comment list with create/delete-own-comment, author identity, timestamps, and reporting. Add threaded replies later if scope permits.

6. Profile

- Editable display name, profile image, bio, age, and gender selection restricted to Male or Female as requested.
- Validate age as a number; store age rather than full birth date unless exact date is genuinely needed.
- Upload profile image to Firebase Storage with progress and retry.
- Profile shows the user's posts and basic engagement counts; avoid vanity leaderboards.
- Provide account deletion, sign-out, and profile visibility options.
- Do not request contacts, microphone, location, or camera unless a shipped feature truly needs it.

7. Firebase Data Model

Use Cloud Firestore for structured data and real-time listeners, Firebase Storage for image bytes, Firebase Authentication for identity, and FCM for push notifications.

Suggested collections

```
users/{uid}
uid, displayName, photoURL, bio, age, gender, createdAt, updatedAt, themePreference, notificationPreferences,
accountStatus

posts/{postId}
postId, authorId, authorNameSnapshot, authorPhotoSnapshot, text, imageURL, imageStoragePath, imageCaption,
topics[], createdAt, updatedAt, likeCount, commentCount, visibility, status

posts/{postId}/likes/{uid}
uid, createdAt
```

users/{uid}/savedPosts/{postId}
postId, savedAt

posts/{postId}/comments/{commentId}
commentId, authorId, text, createdAt, updatedAt, status

users/{uid}/notifications/{notificationId}
type, actorId, postId, createdAt, readAt, deliveryStatus

users/{uid}/blocks/{blockedUid} and **reports/{reportId}**
Store blocking relationships and moderation reports with restricted access.

Treat counts as derived or server-maintained. Use trusted Cloud Functions or Firestore transactions for count updates; never trust client-supplied counts. Avoid unbounded arrays of likes/comments/saved IDs in one document.

8. Security & Backend Rules

- Require Firebase Authentication for writes.
- A user can edit their own profile and delete their own posts; clients cannot change authorId or moderation-only fields.
- Validate fields and types in Firestore Security Rules; enforce image size/content-type limits in Storage Rules.
- Restrict notification reads to the recipient. Restrict report visibility to authorized moderation infrastructure.
- Prevent users from creating/deleting likes on behalf of another uid.
- Use Cloud Functions for trusted notifications and sensitive counters. Never embed service-account credentials in the app.
- Provide tested Firestore and Storage rules plus emulator tests before release.
- Add abuse controls: reporting, blocking, validation, and rate limits where feasible.

9. Visual Design System

Core palette

- Museum Cream #F7F2E8 — primary background.
- Warm Paper #FFFDF8 — cards and elevated surfaces.
- Ink #29251F — primary text; Muted Taupe #81786D — secondary text.
- Gallery Sage #77846F; Aged Gold #B38B58; Clay Rose #C98779.
- Watercolor Blue #91AFC0; Lavender #B0A4C7; Terracotta #B86E52.
- Midnight #24282B for dark mode, with contrast-adjusted surfaces and text.

UI rules

- Use soft SVG/vector icons or consistent Material Symbols with matching stroke weight.
- Use elegant serif typography sparingly for titles and readable sans-serif type for body and controls.
- Use restrained rounded cards, thin borders, and minimal shadows; artwork is the visual focus.
- Use subtle motion and haptics without delaying actions.
- Offer Cream, Sage, Watercolor, Rose, Lavender, Terracotta, and Midnight themes; persist the choice locally and optionally sync it to the profile.
- Respect system dark mode, dynamic font size, and accessible contrast in every theme.
- App icon: a circular cream medallion with a simple painted arch/palette/flower motif. No rectangular motif inside the artwork. Android launchers may mask the outer icon, so keep the central emblem circular.

10. Permissions & Notifications

Request permissions contextually, only when a feature needs them. Do not request every permission at first launch.

- Use Android Photo Picker where available; it usually avoids broad gallery permission.
- Request POST_NOTIFICATIONS on Android 13+ after explaining the benefit.

- Request camera permission only if in-app photo capture is implemented.
- Avoid MANAGE_EXTERNAL_STORAGE (“all files access”): it is not needed for a social art app and is highly restricted by Google Play.
- Implement FCM token registration/refresh and trusted push delivery through Cloud Functions or another secure backend.
- The in-app notification list must work even if push permission is denied; respect user preferences.

11. Real-Time Behavior & Quality

- Feed updates arrive through Firestore listeners without manual refresh; pagination and listener lifecycles stay efficient.
- Likes, saves, comments, profile edits, and notification read states persist in Firebase.
- Use loading skeletons, empty states, offline/error banners, retry actions, and optimistic UI only with rollback on failure.
- Handle Firestore offline persistence intentionally and show pending writes clearly.
- Test slow networks, duplicate taps, upload cancellation, expired sessions, denied permissions, and missing images.
- Use Coil caching and correctly sized image variants; avoid loading original-resolution artwork in every feed card.
- Add repository/ViewModel unit tests, UI tests for key journeys, and Firebase Emulator security-rule tests.

12. MVP Delivery Plan

Phase 1 — Foundation

- Project setup, design tokens, navigation, authentication, session restore, profile setup.

Phase 2 — Social core

- Firestore models/rules, create post, image upload, feed pagination/listeners, likes, saved posts, post detail.

Phase 3 — Community

- Comments, notifications, profile editing, theme preferences, report/block, account deletion.

Phase 4 — Release readiness

- Accessibility, offline/error handling, security-rule tests, performance, privacy disclosures, and real-device testing.

Definition of done: no placeholder actions, hardcoded fake users/posts in production, simulated likes, pretend upload progress, or fake notification records. Empty states are acceptable; demo content must be isolated to development builds.

13. Copy-Ready Build Prompt

```
Build ARTMUSEUM, a complete native Android social app using Kotlin and Jetpack Compose. Application ID: com.artmuseum.app.

PRODUCT: A calm, art-inspired social platform for real people to share honest thoughts about life, relationships, courage, personal growth, peace, and art. Users publish long-form text posts with optional artwork images and captions, read a real-time feed, like/unlike, save/unsave, comment, view author profiles, edit their profile and posts, and receive activity notifications.

STACK: Kotlin, Jetpack Compose, Material 3 with a custom theme system, MVVM + repositories, Coroutines/Flow, Navigation Compose, Hilt where compatible, Firebase Authentication, Cloud Firestore, Firebase Storage, Firebase Cloud Messaging, Coil, Android Photo Picker, and Firebase Emulator Suite. Use current stable compatible versions and document them.

AUTH: Google sign-in and email/password registration/login, password reset, session restore, clear error/loading states, and profile setup for new users. Never simulate successful auth.

PROFILE: display name, profile photo, bio, age, and Male/Female gender selection. Allow editing, own-post list, sign-out, privacy options, and account deletion. Store age rather than full birth date unless necessary.

POSTS: no artificial word limit in the editor; support text-only posts and optional artwork image, caption, and topic tags. Upload images to Firebase Storage and persist metadata in Firestore. Include progress, retry, cancel, edit/delete own posts, real-time feed, pagination, post details, likes, saves, comments, and proper empty/error/offline states. All actions must use real backend calls.

BACKEND: Firestore collections for users, posts, nested likes/comments, user savedPosts, user notifications,
```

```
blocks, and reports. Use secure Firestore and Storage rules, Cloud Functions for trusted notification
delivery and sensitive counters, ownership checks, validation, and emulator tests. Never ship service-account
credentials in the Android client.

DESIGN: cream-first (#F7F2E8) museum aesthetic, warm paper cards, ink text, sage, aged gold, clay rose,
watercolor blue, lavender, terracotta, and Midnight themes. Minimal, compact, premium, artful UI with soft
SVG/vector icons, elegant serif headings used sparingly, readable sans-serif body type, restrained shadows,
rounded cards, subtle transitions, accessible contrast, and responsive layouts for all phone sizes. Bottom
navigation: Home, Explore, Create, Saved, Profile. Notifications accessible from the top bar. Provide
light/dark mode and persistent theme selection. Design a circular app emblem with no rectangular motif inside
it.

PERMISSIONS: request only when needed. Use Android Photo Picker for images; request POST_NOTIFICATIONS on
Android 13+ after context. Do not request broad all-files access for this app. Implement FCM and an in-app
notification center.

SAFETY: report post/comment/user, block user, delete own content, account deletion, and notification/privacy
preferences. Protect personal information. Provide rules and tests.

QUALITY: no fake feeds, fake users, mock likes, simulated uploads, or placeholder controls in production.
Implement loading, retry, validation, network failure handling, optimistic updates with rollback,
lifecycle-safe listeners, pagination, accessibility, and real-device tests. Deliver the complete runnable
Android project, Firebase setup instructions, Firestore/Storage rules, Cloud Functions source if used,
emulator tests, and README. Build and verify a debug APK; clearly list any Firebase console configuration the
developer must complete.
```

14. Release Checklist

- Google and email sign-in tested on a real device.
- Firestore and Storage rules deployed and tested with emulator cases.
- Image uploads, retries, deletion cleanup, and image loading verified.
- Likes and saves persist across app restarts and account sessions.
- Feed and notifications show actual backend data.
- Report/block and account deletion tested.
- Permission requests are contextual and have graceful denied states.
- Circular icon and theme palette checked at launcher and phone sizes.
- Privacy policy, data-safety disclosures, and account deletion route prepared before store release.

Pill-Shaped UI Components & Rounded-Corner Language

This amendment extends the original Artmuseum Kotlin full-stack PRD. Apply it throughout the app and to the copy-ready build prompt. These requirements are part of the core design system, not optional decoration.

1. Core shape language

- **Pill-shaped controls:** buttons, category filters, topic tags, segmented tabs, theme selectors, small action chips, and selected filters should use fully rounded capsule ends (Compose shape equivalent to a large/full pill radius).
- **Rounded containers:** artwork cards, post surfaces, profile panels, sheets, and image previews should use consistently rounded corners. Use a softer, smaller radius than pills so controls and containers remain visually distinct.
- **Circle-only elements:** avatars, icon-only action buttons when appropriate, radio indicators, and the Artmuseum emblem should remain circular. The app icon must feature a circular motif without a rectangular shape inside it.
- Keep the cream-first museum palette, painterly artwork, subtle borders, low-elevation shadows, soft SVG/vector icons, and uncluttered mobile layouts.
- Do not make every surface a pill. Pills are for compact controls; cards are for grouped content; circles are for avatars and icon actions.

2. Component specifications

- **Primary buttons:** full-width where appropriate, pill radius, cream/ink or sage/cream contrast, comfortable height around 48–54 dp, clear pressed/disabled/loading states.
- **Secondary buttons:** pill-shaped outline or warm-paper fill with a thin muted border; avoid heavy shadows.
- **Filter chips:** For You, Life, Love, Courage, Growth, Peace, Art and similar categories use compact pill chips. Selected state uses a restrained sage/ink fill; unselected state uses warm paper or transparent cream with a subtle outline.
- **Topic tags:** Life, Love, Courage, Growth and Peace appear as small rounded pills in the composer and post details. Keep labels short and readable.
- **Segmented controls:** Posts/Saved and theme or feed mode selectors use a pill-shaped track with a clearly contrasting pill-shaped selected segment.
- **Text fields:** email, password, caption, bio, and search fields use rounded corners with soft borders. Short fields may be capsule-shaped; long-form post and bio editors should use rounded rectangles, not narrow pills.
- **Icon buttons:** use circular hit areas for search, notifications, bookmark, like, close, and overflow when displayed alone. Use soft vector icons with consistent line weight and accessible touch targets.
- **Comment composer:** rounded pill input anchored above navigation where relevant, with a separate circular send button.
- **Theme picker:** display each palette as a circular color swatch; selection should have a visible ring/check mark. Theme names can appear in a pill label or compact row.
- **Permission and onboarding actions:** Google and Email sign-in buttons should be large pill-shaped controls with clear labels and familiar provider icons.

3. Compose implementation guidance

Create reusable composables instead of drawing each pill separately. Keep the radius, spacing, color, and interaction rules in a central theme/design-token file.

```
Suggested component inventory: ArtmuseumPrimaryButton ArtmuseumSecondaryButton ArtmuseumIconButton
ArtmuseumFilterChip ArtmuseumTopicChip ArtmuseumSegmentedControl ArtmuseumTextField
ArtmuseumSearchField ArtmuseumPostCard ArtmuseumArtworkCard ArtmuseumCommentComposer
ArtmuseumThemeSwatch ArtmuseumBottomNavigation ArtmuseumPermissionRow
```

Use Material 3 shapes or custom Shape values with consistent tokens. A full pill radius can be represented by a very large radius (for example, CircleShape for capsule controls); choose CircleShape only where it produces the intended

capsule silhouette. Use rounded shapes for multi-line fields and content cards. Ensure Compose previews cover small and large phones, long labels, accessibility font scaling, and light/dark themes.

4. Suggested shape and spacing tokens

- **Pill:** CircleShape / maximum radius for single-line controls and chips.
- **Small rounded:** approximately 12 dp for compact input fields and small surfaces.
- **Card:** approximately 18–24 dp for posts, artwork, and grouped panels.
- **Large sheet:** approximately 24–30 dp on top corners for bottom sheets and dialogs.
- **Spacing:** use a consistent 4/8 dp spacing rhythm; keep controls thumb-friendly and avoid cramped chips.
- These are design tokens, not rigid platform constraints. Maintain visual consistency and fit the content to each phone width.

5. Interaction and accessibility

- All pills need distinct default, pressed, selected, disabled, loading, and focus states where relevant.
- Use animated color/scale transitions subtly; do not animate layout in ways that cause content to jump.
- Support touch targets around 48 dp even when the visible icon or chip is smaller.
- Long category labels should scroll horizontally or wrap safely rather than overflow off-screen.
- Selected state must not rely on color alone; use contrast, shape, text weight, or a check indicator.
- Respect system font scaling, screen readers, and dark mode. Validate contrast for every custom theme.
- Avoid excessive nested pills, thick outlines, glossy gradients, and decorative glass effects that compete with artwork.

6. Updated acceptance criteria

- Every main action and sign-in action uses the defined pill-shaped component family.
- Feed filters, topic tags, segmented tabs, and theme controls use consistent rounded patterns.
- Post cards and image previews have soft rounded corners; avatars and icon-only controls remain circular where appropriate.
- No UI element clips or overflows on compact Android phones or at enlarged font sizes.
- The same reusable Compose components are used across authentication, feed, post detail, creation, saved posts, profile, and settings.
- The original cream-first palette, soft art-inspired themes, minimalist layout, and circular app-icon requirement remain intact.